

HW04 - Automation Testing Report

Student: 23127459 - Group 06

SUT: EShop (E-Commerce Platform)

Tool: Playwright + TypeScript (Data-Driven, Multi-Browser)

Report Run by: 23127459

Video Demo: <https://youtu.be/bx-9XmGdQYA>

Link repo github thực hiện: https://github.com/iamDicun/Group06_HW2_Testing/tree/HW04-23127459

Nhánh: HW04-23127459

1. Execution Summary

1.1. Test Environment

Component	Technology	URL
Backend API	Node.js + Express + SQLite	http://localhost:3000
Frontend Web	React + Vite + Tailwind CSS	http://localhost:5173
Frontend Admin	React + Vite + Tailwind CSS	http://localhost:5174
Test Framework	Playwright v1.62.1 + TypeScript	-

1.2. Multi-Browser Configuration

Browser	Device	Status
Chromium	Desktop Chrome	Executed
Firefox	Desktop Firefox	Executed
WebKit	Desktop Safari	Executed

1.3. Test Execution Summary

Total Features: 3 **Total Test Cases:** 51 (FR-05: 20, FR-11: 17, FR-17: 14) **Total Runs:** 153 (51 TCs x 3 browsers)

Metadata Verification: Run by: 23127459 confirmed in `playwright.config.ts` reporter title and `testInfo.annotations`

Feature	Feature Name	Test Cases	Runs (x3 browsers)	Pass	Fail	Status
FR-05	Product Listing & Search	20	60	29	31	FAIL

Feature	Feature Name	Test Cases	Runs (x3 browsers)	Pass	Fail	Status
FR-11	Order History View (User)	17	51	45	6	FAIL
FR-17	Coupon Management (CRUD Admin)	14	42	42	0	ALL PASS
TOTAL		51	153	116	37	

FR-05 FAILURE ROOT CAUSE: The original test assertions were **too weak** (false positives). After refactoring with stricter assertions, 10 out of 20 FR-05 test cases now correctly **FAIL**, revealing real SUT bugs that the original tests missed.

1.4. Test Coverage Matrix

FR-05: Product Listing & Search (20 TCs)

TC ID	Test Case Description	Type	Chromium	Firefox	WebKit
FR05-TC001	Page title "Danh sách sản phẩm"	Positive	PASS	PASS	PASS
FR05-TC002	Exactly 1 <h1> tag (FR-05/FR-21)	SUT BUG	FAIL	FAIL	FAIL
FR05-TC003	Grid displays >= 1 product	Positive	PASS	PASS	PASS
FR05-TC004	Product card: name, price, image	Positive	PASS	PASS	PASS
FR05-TC005	Price has thousands separator	Positive	PASS	PASS	PASS
FR05-TC006	Price uses "₫" not "VND" (FR-05/FR-21)	SUT BUG	FAIL	FAIL	FAIL
FR05-TC007	Search exact name returns 1 result	Positive	PASS	PASS	PASS
FR05-TC008	Search "Pro" returns >= 1 result	Positive	PASS	PASS	PASS
FR05-TC009	Case-insensitive search	Edge	PASS	PASS	PASS
FR05-TC10	Non-existent term returns 0	Negative	PASS	PASS	PASS
FR05-TC11	Empty state message (FR-05)	SUT BUG	FAIL	FAIL	FAIL
FR05-TC12	XSS test not rendered (SEC-04)	SUT BUG	FAIL	FAIL	FAIL

TC ID	Test Case Description	Type	Chromium	Firefox	WebKit
FR05-TC13	Alt text not empty (FR-05/FR-24)	SUT BUG	FAIL	FAIL	FAIL
FR05-TC14	Search keyword shown in results	Positive	PASS	PASS	PASS
FR05-TC15	Exact "₫" symbol, no "VND" (FR-05/FR-21)	SUT BUG	FAIL	FAIL	FAIL
FR05-TC16	All images have descriptive alt (FR-05/FR-24)	SUT BUG	FAIL	FAIL	FAIL
FR05-TC17	XSS <script> not rendered (SEC-04/FR-05)	SUT BUG	FAIL	FAIL	FAIL
FR05-TC18	No duplicate <h1> tags (FR-05/FR-21)	SUT BUG	FAIL	FAIL	FAIL
FR05-TC19	Price format "XX,XXX,XXX ₫" (FR-05/FR-21)	SUT BUG	FAIL	FAIL	FAIL
FR05-TC20	Image src is valid URL	Edge	PASS	FAIL	PASS

FR-11: Order History View (17 TCs)

TC ID	Test Case Description	Type	Chromium	Firefox	WebKit
FR11-TC001-TC010	Order history display (10 TCs)	Positive	PASS	PASS	PASS
FR11-TC011	Status badge color classes	SUT BUG	FAIL	FAIL	FAIL
FR11-TC012	Newest orders first	Positive	PASS	PASS	PASS
FR11-TC013	No cancel for delivered/canceled	Positive	PASS	PASS	PASS
FR11-TC014	No cancel for "shipping" (FR-10)	SUT BUG	FAIL	FAIL	FAIL
FR11-TC015-TC017	Own orders, empty, login required	Mixed	PASS	PASS	PASS

FR-17: Coupon Management CRUD (14 TCs)

TC ID	Test Case Description	Type	Chromium	Firefox	WebKit
FR17-TC001-TC014	All coupon CRUD operations	Mixed	PASS	PASS	PASS

2. Human Review & Gap Analysis (Script Quality)

2.1. False Positive Analysis — Why Original AI Assertions Were Too Weak

The original AI-generated FR-05 tests had **6 critical assertion weaknesses** that caused false positives (tests passed when the SUT had real bugs). The refactored tests now correctly detect these bugs.

Weakness 1: `h1:visible` Filter Masked Duplicate `<h1>` Tags

Original (WRONG):

```
await expect(page.locator("h1:visible")).toHaveCount(1);
```

Problem: The SUT has 2 `<h1>` tags. Both are visible. But `h1:visible` with `.toHaveCount(1)` may have passed due to timing — the second `<h1>` renders after products load. The original test checked too early.

Refactored (CORRECT):

```
const allH1 = page.locator("h1");
const h1Count = await allH1.count();
expect(h1Count, `SUT co ${h1Count} the <h1>`).toBe(1);
```

Why this catches the bug: No `:visible` filter. Counts ALL `<h1>` in DOM. The SUT has `<h1>Danh sách sản phẩm</h1>` + `<h1>Hiển thị 5 sản phẩm</h1>` = 2 tags → FAIL.

Weakness 2: `toContainText("₫")` Passed for "VND"

Original (WRONG):

```
await expect(card).toContainText("₫");
```

Problem: `toContainText` is a **substring match**. The SUT renders `"30,000,000 VND"`. This does NOT contain `"₫"`, so it should fail. However, the original test used `card` (the entire product card) which includes multiple `<p>` elements. If any text matched, it passed. The real issue: the assertion was on the wrong element.

Refactored (CORRECT):

```
const priceText = await home.priceElements.first().textContent();
expect(priceText).toContain("₫");
```

Why this catches the bug: Targets the exact `<p class="text-red-500">` element. `"30,000,000 VND"` does not contain `"₫"` → FAIL.

Weakness 3: XSS Check Only Counted `<b>` Tags

Original (WRONG):

```
await expect(home.resultLine.locator("b")).toHaveCount(0);
```

Problem: The SUT uses `dangerouslySetInnerHTML={{ __html: search }}`. When searching `<b>test</b>`, the `<b>` tag IS rendered. But the original assertion only checked `<b>` count — if the SUT rendered it differently (e.g., as `<strong>`), it would pass as a false positive.

Refactored (CORRECT):

```
const boldCount = await home.resultLine.locator("b").count();
expect(boldCount, `XSS: tim kiem da render ${boldCount} the <b>`).toBe(0);
// PLUS: Check full body HTML for script injection
const bodyHtml = await page.locator("body").innerHTML();
expect(bodyHtml.includes("<script>")).toBe(false);
expect(bodyHtml.includes("alert(")).toBe(false);
```

Why this catches the bug: Both checks run. The `<b>` count check catches `<b>test</b>`. The body HTML check catches `<script>alert('xss')</script>`.

Weakness 4: Alt Check Used `toHaveAttribute("alt")` Which Passes for Empty String

Original (WRONG):

```
await expect(img).toHaveAttribute("alt");
const altValue = await img.getAttribute("alt");
expect(altValue?.trim()).not.toBe("");
```

Problem: `toHaveAttribute("alt")` passes even when `alt=""` (attribute exists, value is empty). The second check `not.toBe("")` should have caught this, but it was in a `for` loop that may have short-circuited.

Refactored (CORRECT):

```
const altValue = await img.getAttribute("alt");
expect(altValue?.trim(), `Anh #${i}: alt PHAI co text mo ta`).not.toBe("");
```

Why this catches the bug: Directly checks that alt text is not empty. SUT has `alt=""` → `"".trim() === ""` → FAIL.

Weakness 5: Missing Price Format Regex Check

Original: No regex check for price format.

Refactored (NEW):

```
const regex = new RegExp("^[\\d,]+ đ$");
expect(priceText).toMatch(regex);
```

Why this catches the bug: "30,000,000 VND" does not match `/^[\\d,]+ đ$/` → FAIL.

Weakness 6: No Test for `<script>` Tag Injection

Original: Only tested `<b>` tag, not `<script>`.

Refactored (NEW):

```
await page.waitForLoadState("networkidle");
const bodyHtml = await page.locator("body").innerHTML();
expect(bodyHtml.includes("<script>")).toBe(false);
expect(bodyHtml.includes("alert(")).toBe(false);
```

Why this catches the bug: SUT renders `<script>alert('xss')</script>` via `dangerouslySetInnerHTML` → body contains `<script>` and `alert(` → FAIL.

2.2. Summary of Refactoring

Issue	Category	Before (Weak)	After (Strict)	Bug Caught
h1 count	False positive	<code>h1:visible</code> count	All <code>h1</code> count	2 <code><h1></code> tags
Currency symbol	False positive	<code>toContainText("đ")</code> on card	<code>textContent()</code> on price element	"VND" not "đ"
XSS <code></code>	Partial check	Count <code></code> only	Count <code></code> + body HTML check	<code>dangerouslySetInnerHTML</code>
Image alt	False positive	<code>toHaveAttribute("alt")</code>	<code>altValue.trim() !== ""</code>	<code>alt=""</code> empty
Price format	Missing	None	Regex <code>/^[\\d,]+ đ\$/</code>	Wrong format
<code><script></code> XSS	Missing	None	<code>bodyHtml.includes("<script>")</code>	Script injection

3. Bug Report (GitHub Issue Format)

BUG-001: Homepage Renders 2 `<h1>` Tags (Violates FR-05/FR-21)

Title: `[BUG]` Homepage renders 2 `<h1>` tags - violates FR-05/FR-21

Environment:

- SUT: EShop Frontend Web (`http://localhost:5173`)
- Browser: All (Chromium, Firefox, WebKit)
- OS: Windows 11

Severity: Medium | **Priority:** P2 **Feature:** FR-05 | **Related Spec:** FR-05/FR-21

Steps to Reproduce:

1. Navigate to `http://localhost:5173`
2. Open browser DevTools → Console
3. Run: `document.querySelectorAll('h1').length`

Expected Result: 1 (FR-05: "Trang chủ chỉ có đúng một thẻ `<h1>`")

Actual Result: 2

- `<h1>Danh sách sản phẩm</h1>` (line 43 of Home.jsx)
- `<h1>Hiển thị 5 sản phẩm</h1>` (line 110 of Home.jsx)

Evidence:

- Screenshot: `test-results/FR-05-FR-05-Xem-danh-sach--d834c--h1-theo-dac-ta-FR-05-FR-21-webkit/test-failed-1.png`
- GitHub Issue Screenshot: `Bug_01_GhubIssue.png`
- Trace: `test-results/FR-05-FR-05-Xem-danh-sach--d834c--h1-theo-dac-ta-FR-05-FR-21-webkit/trace.zip`

Suggested Fix:

```
<!-- BEFORE (Home.jsx line 110): -->
<h1 className="text-center text-gray-400 mt-8 text-sm">
  Hiển thị {products.length} sản phẩm
</h1>

<!-- AFTER: -->
<p className="text-center text-gray-400 mt-8 text-sm">
  Hiển thị {products.length} sản phẩm
</p>
```

BUG-002: Price Displays "VND" Instead of "₫" (Violates FR-05/FR-21)

Title: [BUG] Price shows "VND" text instead of "₫" symbol - violates FR-05/FR-21

Environment:

- SUT: EShop Frontend Web (`http://localhost:5173`)
- Browser: All

Severity: Medium | **Priority:** P2 **Feature:** FR-05 | **Related Spec:** FR-05/FR-21

Steps to Reproduce:

1. Navigate to `http://localhost:5173`
2. Observe any product price

Expected Result: Price displays with "₫" symbol (e.g., "30,000,000 ₫")

Actual Result: Price displays "VND" text (e.g., "30,000,000 VND")

Evidence:

- Screenshot: [test-results/FR-05-FR-05-Xem-danh-sach--2bc42-g-₫-theo-dac-ta-FR-05-FR-21-webkit/test-failed-1.png](#)
- GitHub Issue Screenshot: [Bug_02_GhubIssue.png](#)

Suggested Fix:

```
// BEFORE (Home.jsx line 87):  
<p className="text-red-500 font-bold mb-2">  
  {Number(p.price).toLocaleString()} VND  
</p>  
  
// AFTER:  
<p className="text-red-500 font-bold mb-2">  
  {Number(p.price).toLocaleString()} ₫  
</p>
```

BUG-003: Product Images Have Empty `alt=""` Attribute (Violates FR-05/FR-24)

Title: [BUG] Product images have empty `alt=""` - violates FR-05/FR-24 accessibility

Environment:

- SUT: EShop Frontend Web (<http://localhost:5173>)
- Browser: All

Severity: Medium | **Priority:** P2 **Feature:** FR-05 | **Related Spec:** FR-05/FR-24

Steps to Reproduce:

1. Navigate to <http://localhost:5173>
2. Right-click any product image → Inspect
3. Check `alt` attribute

Expected Result: `alt="iPhone 15 Pro Max"` (descriptive text per FR-24)

Actual Result: `alt=""` (empty string)

Evidence:

- Screenshot: [test-results/FR-05-FR-05-Xem-danh-sach--f538f--tinh-alt-khong-rong-FR-24-webkit/test-failed-1.png](#)
- GitHub Issue Screenshot: [Bug03_GhIssue.png](#)

Suggested Fix:

```
// BEFORE (Home.jsx line 80-84):  
<img src={p.imageUrl} alt="" className="w-full h-48 object-cover mb-4 rounded" />  
  
// AFTER:  
<img src={p.imageUrl} alt={p.name} className="w-full h-48 object-cover mb-4  
rounded" />
```

BUG-004: XSS via `dangerouslySetInnerHTML` in Search Results (Violates SEC-04)

Title: [BUG] Search uses `dangerouslySetInnerHTML` - XSS vulnerability (SEC-04)

Environment:

- SUT: EShop Frontend Web (<http://localhost:5173>)
- Browser: All

Severity: High | **Priority:** P1 **Feature:** FR-05 | **Related Spec:** SEC-04

Steps to Reproduce:

1. Navigate to <http://localhost:5173>
2. In search box, type: `<b>test</b>`
3. Click "Tìm"
4. Observe: "Kết quả tìm kiếm cho:" section renders `<b>test</b>` as bold text

Expected Result: Search term displayed as plain text: `<b>test</b>` (no HTML rendering)

Actual Result: `<b>test</b>` renders as **bold text** "test"

Evidence:

- Screenshot: [test-results/FR-05-FR-05-Xem-danh-sach--a8c4b-n-khong-render-HTML-SEC-04--webkit/test-failed-1.png](#)
- GitHub Issue Screenshot: [Bug04_GhIssue.png](#)

Suggested Fix:

```
// BEFORE (Home.jsx line 63-65):  
<span dangerouslySetInnerHTML={{ __html: search }} />  
  
// AFTER:  
<span>{search}</span>
```

BUG-005: No Empty State Message Displayed (Violates FR-05)

Title: [BUG] No empty state message when search returns 0 results - violates FR-05

Environment:

- SUT: EShop Frontend Web (<http://localhost:5173>)
- Browser: All

Severity: Low | Priority: P3 Feature: FR-05

Steps to Reproduce:

1. Navigate to <http://localhost:5173>
2. Search for "zzzzzzzzzz" (non-existent term)
3. Observe the page

Expected Result: Empty state message like "Không tìm thấy sản phẩm" displayed

Actual Result: Empty grid, no message

Evidence:

- Screenshot: [test-results/FR-05-FR-05-Xem-danh-sach--c7861--empty-state-phu-hop-FR-05--webkit/test-failed-1.png](#)
- GitHub Issue Screenshot: [Bug05_GhIssue.png](#)

Suggested Fix:

```
// Add after the grid in Home.jsx:
{products.length === 0 && search && !errorHtml && (
  <p className="text-center text-gray-500 mt-8">Không tìm thấy sản phẩm</p>
)}
```

4. AI Critique (200-300 words)

The original AI-generated FR-05 test suite contained **systematic false positives** caused by assertion patterns that were too lenient. The most critical failure was in **assertion specificity** — the AI used `toContainText("₫")` on the entire product card instead of targeting the price element directly, and `toHaveAttribute("alt")` which passes for empty strings. These weak assertions created a dangerous illusion of correctness: 42 out of 42 FR-05 tests passed on all 3 browsers, yet the SUT had at least 5 real bugs.

The root cause was that the AI **tested against specs rather than reality**. It wrote assertions assuming the SUT complied with FR-05 ("dùng ký hiệu ₫"), without verifying what the frontend actually renders. The AI never inspected `Home.jsx` to discover that the price element uses `"VND"` text, that `alt=""` is empty, or that `dangerouslySetInnerHTML` creates XSS vectors. This reveals a fundamental limitation: AI generates tests from *specifications*, but specifications describe *desired behavior*, not *actual implementation*.

After human review identified these gaps, I refactored with **targeted selectors** (`home.priceElements.first().textContent()` instead of `card`), **explicit value checks** (`altValue.trim() !== ""` instead of `toHaveAttribute("alt")`), and **regex pattern matching** (`/^[\\d,]+ ₫$/` for price format). The refactored tests immediately caught 10 SUT bugs across 3 browsers that the original tests completely missed. The lesson: AI excels at generating test *structure* and *coverage matrices*, but human

engineers must validate assertion *precision* — the difference between a test that runs and a test that actually catches bugs.

5. AI Audit Log Report (Appendix)

Declaration: "I use AI tools for the following tasks"

#	AI Tool	Date & Time	Prompt Input	AI Output Summary
1	OpenAI ChatGPT (GPT-4)	2026- 08-08 10:00	Design Playwright test framework for EShop FR-05, FR-11, FR-17 with DDT, multi-browser, page objects	Generated complete project structure: playwright.config.ts, 3 spec files, page objects, data loader utility, fixtures, test data JSON files
2	OpenAI ChatGPT (GPT-4)	2026- 08-08 11:30	Create test data JSON files for FR-05 (14 cases), FR-11 (17 cases), FR-17 (14 cases) with positive, negative, edge cases	Generated fr05-data.json, fr11-data.json, fr17-data.json with data-driven test cases including timestamp substitution
3	OpenAI ChatGPT (GPT-4)	2026- 08-08 14:00	Write page object models for HomePage, LoginPage, ProfilePage, AdminLoginPage, AdminCouponsPage with Playwright locators	Generated 5 page object classes with typed locators and helper methods
4	OpenAI ChatGPT (GPT-4)	2026- 08-08 15:30	Create API seeding utility for test data setup - register users, create orders, set order status via API	Generated apiSeed.ts with functions for user registration, login, order creation, and status transitions
5	OpenAI ChatGPT (GPT-4)	2026- 08-09 09:00	Analyze FR-11 test failure - shipping order shows cancel button when it should not	Identified root cause: Frontend renders cancel button for all non-final states, violating FR-10 state machine spec
6	OpenAI ChatGPT (GPT-4)	2026- 08-10 10:00	Read SUT Home.jsx source code and identify false positive causes in FR-05 test assertions	Found 6 weak assertion patterns: h1:visible filter, toContainText on wrong element, toHaveAttribute for empty alt, XSS check incomplete, missing price regex, missing script tag check
7	OpenAI ChatGPT (GPT-4)	2026- 08-10 11:00	Refactor FR-05 spec.ts with stricter assertions to catch SUT bugs: empty alt, VND currency, XSS, duplicate h1	Rewrote FR-05.spec.ts with 20 test cases (6 new), added priceElements locator, regex price check, body HTML XSS check, alt text empty check
8	OpenAI ChatGPT (GPT-4)	2026- 08-10 14:00	Run Playwright on 3 browsers and generate bug reports for FR-05 SUT bugs found	Tests ran: 37 failed / 116 passed. Generated 5 bug reports in GitHub Issue format with evidence paths